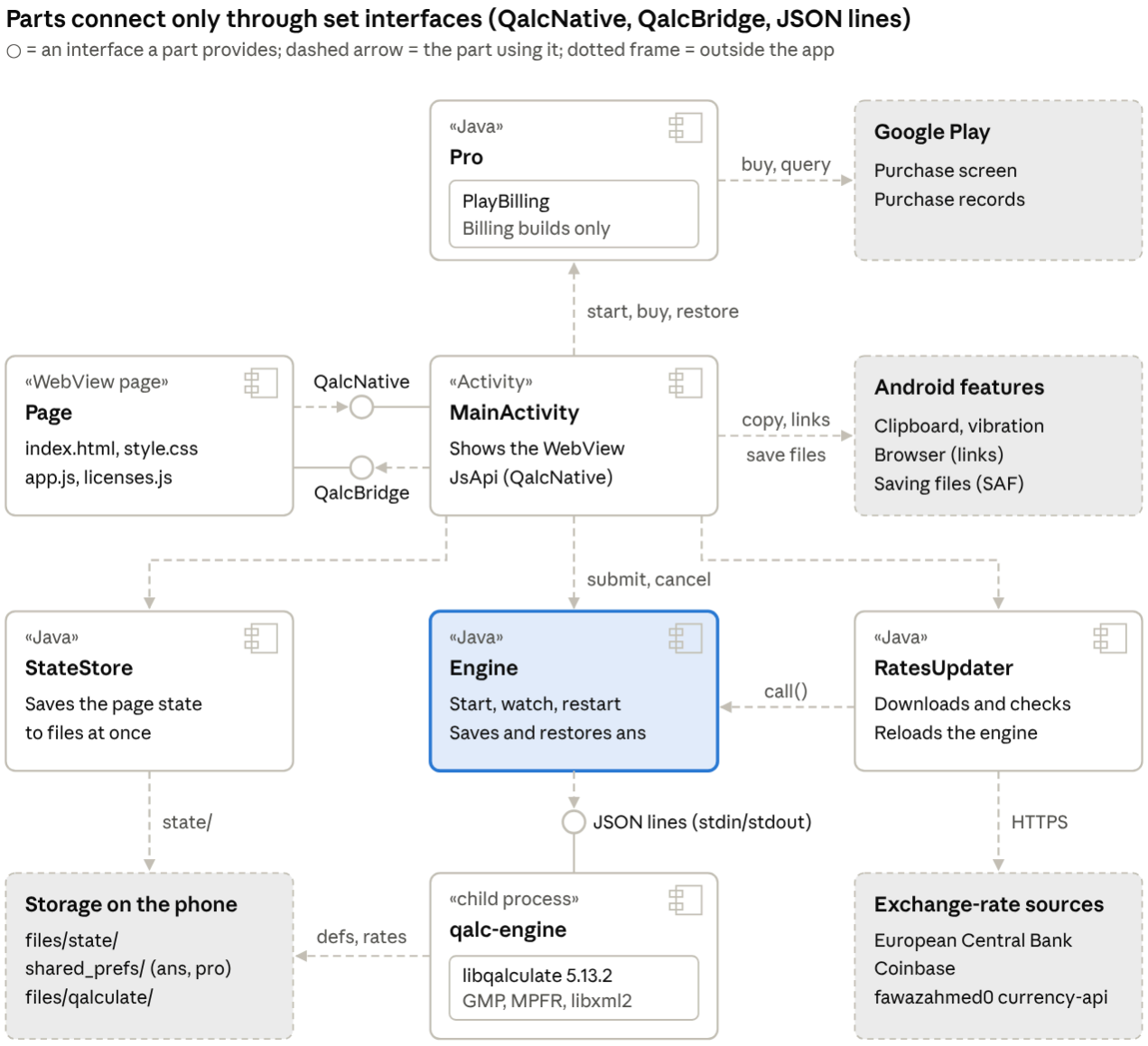
2. Basic design

QCalc runs in three layers: the page (a WebView), the Java code of the Android app, and a separate process with the official calculation library libqalculate. It is distributed on Google Play as an App Bundle (.aab).

2.1 System structure



Component diagram · parts and three interfaces

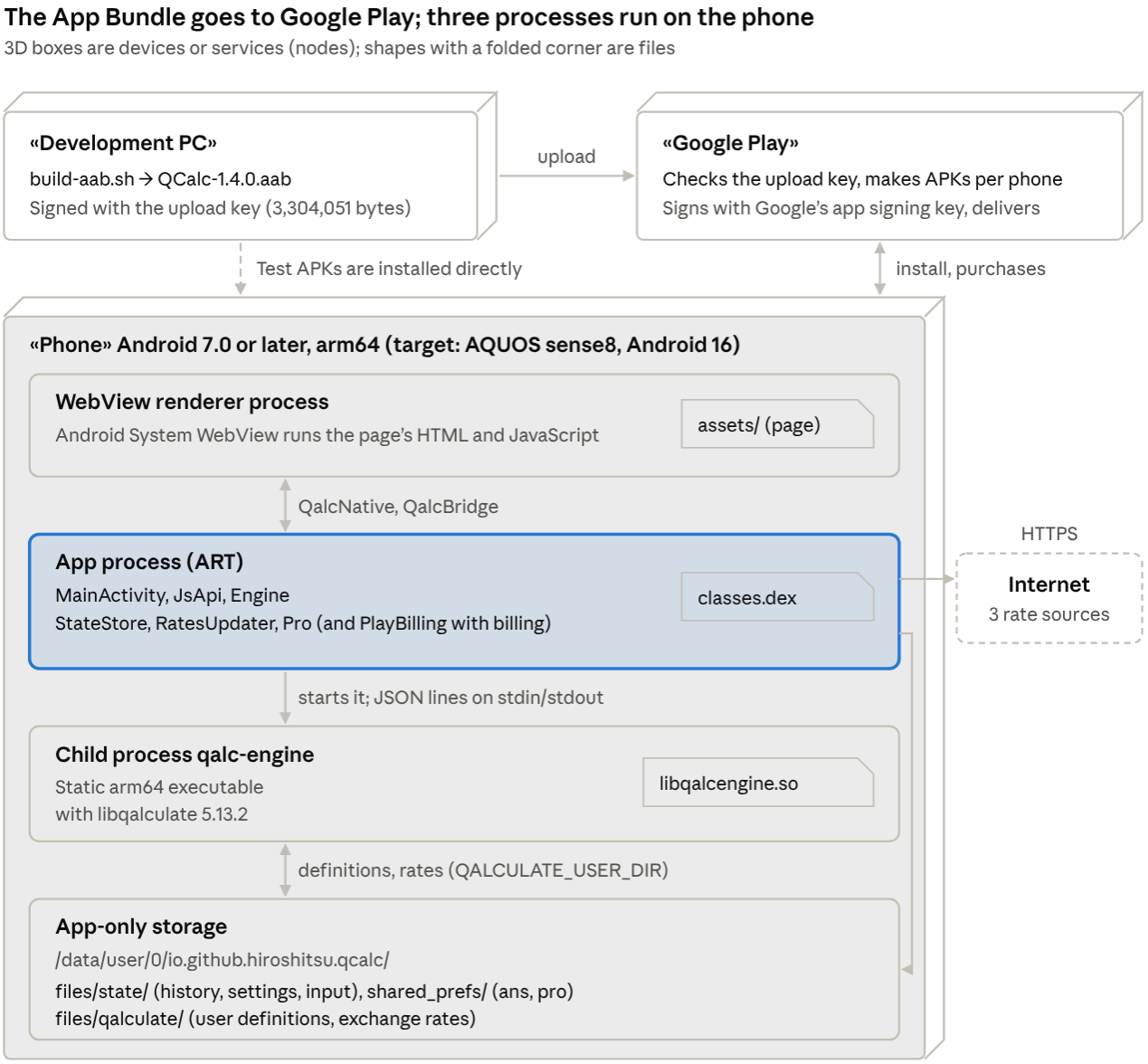
The parts connect only through the three interfaces below. Each passes only strings (JSON), so the page, the Java code and the engine can be tested separately (chapter 6).

Interface	Direction	Contents	Details
QalcNative	Page → Java	The 15 methods JsApi gives the WebView: send, cancel, updateRates, loadState, saveState, copy, appInfo, openUrl, and proInfo, buyPro, restorePro, refreshPro, setTestPro, saveFile added in 1.3.0, and setAppearance added in 1.4.0	3.2
QalcBridge	Java → page	The receiver app.js provides. Java hands the replies to requests to receiveText and the events (ans, and pro and saved added in 1.3.0) to receiveEvent, called with evaluateJavascript	3.2
JSON lines	Engine ⇄ qalc-engine	A request is written to the engine's standard input as one line; replies and events are read from its standard output, one line each	3.3

Requests to the engine, from the page (through Engine's one worker thread) and from the exchange-rate update (RatesUpdater calls call() directly), are sent one at a time, as call() is synchronized. Only cancel does not wait for it: it stops the running engine process.

Pro, added in 1.3.0, is the Java part that holds the state of QCalc Pro; MainActivity creates it first. PlayBilling talks to Google Play. PlayBilling is only in builds with billing (Gradle's qcalc.billing=true), and Pro finds and loads it by its class name. A build without it cannot sell Pro, so it locks nothing (requirements 1.9). When the state of Pro changes, MainActivity tells the page with a pro event.

2.2 Deployment



Deployment diagram · from the App Bundle to the phone, three processes at run time

The App Bundle made by build-aab.sh (QCalc-1.4.0.aab, signed with the upload key) is uploaded to Google Play in the Play Console. Google Play makes APKs with only the parts each phone needs, signs them with the app signing key that Google keeps, and delivers them. For an arm64 phone this is one APK of about 3.4 MB.

At installation, the engine libqalcengine.so is extracted to nativeLibraryDir (extractNativeLibs="true" in the AndroidManifest). Its name ends in .so, but it is an executable, which Engine starts as a child process with ProcessBuilder. Apps with targetSdk 29 or later cannot execute files in their own data, so the engine lives in nativeLibraryDir, where executing is allowed. The App Bundle's settings (BundleConfig) turn off the optimization that keeps native libraries uncompressed; with it on, the APKs Google Play makes would not extract the engine as a file, and it could not start.

Output	Made with	Signature	How it reaches the phone
QCalc-1.4.0.aab (for Google Play, io.github.hiroshitsu.qcalc)	build-aab.sh (without billing), or the release build in Android Studio (with billing when qcalc.billing=true)	The upload key (signing/qcalc-upload.jks). The delivered APKs carry Google's app signing key	Uploaded in the Play Console → review → delivered by Google Play
QCalc-1.4.0-debug.apk (test build, io.github.hiroshitsu.qcalc.debug)	build-apk.sh, or the debug build in Android Studio	The test key (signing/qcalc-debug.keystore)	Installed directly from a file or over USB. It can sit beside the Google Play build

The page's HTML and JavaScript run in the WebView's renderer process (a separate process from Android 8.0). When the renderer stops, MainActivity makes the page again in onRenderProcessGone; if that happens again within 15 seconds, it asks the user to open the app again.

When Engine starts the engine, it passes these environment variables. The working directory is files/.

Variable	Value	Purpose
HOME	files/	The home directory
QALCULATE_USER_DIR	files/qalculate/	Where libqalculate keeps its settings and data (user definitions, exchange rates)

Variable	Value	Purpose
TZ	LOC-9:00 in Japan	The offset from UTC when the engine starts, in POSIX form. The engine is a statically linked Linux program and cannot read Android's time zone data. A change for daylight saving time applies from the engine's next start
LANG	C.UTF-8	Handle text as UTF-8
QALC_CURRENCY	The currency of the phone's region (JPY in Japan)	The local currency. When "Automatic unit conversion" is "Best unit" or "Best SI unit", currency results are converted to it (for example 100 USD → JPY). Not passed when the phone's language setting has no region

2.3 Software structure

The app's own code is 10 files, about 7,200 lines (Pro.java and PlayBilling.java were added in 1.3.0). The calculation core, libqalculate, is the official source built in as a static library.

Layer	Module	File	Lines	Role
Page	Page skeleton	app/assets/index.html	52	The top bar, the history, the input field (with the "Undo" button and the row of suggestions), the containers of the always-shown keys, the tabs, the keys and the menu
Page	Page style	app/assets/style.css	776	Colors (light and dark) and the 7 color themes, keys, key tabs, matrix tables, text sizes, panels, graphs, the landscape layout, suggestion chips, PRO marks

Layer	Module	File	Lines	Role
Page	Page logic	app/assets/app.js	2,918	English and Japanese texts, keys (4 tabs and templates), input, preview, suggestions, calculating, the history (the beginning and the end of long results), matrix tables, undo, examples, lists (Japanese names and search), graphs (2D and 3D), definitions, exchange rates, settings, the QCalc Pro panel and locks, exporting the history (CSV), the licenses
Page	License texts	app/assets/licenses.js	— (generated, about 90 KB)	The names, versions, copyrights and full license texts of the bundled software, made by make-licenses.py
Android	MainActivity	app/src/io/github/hiroshitsu/qcalc/MainActivity.java	622	Creating and setting up the WebView, the JavaScript bridge, screen edges (Android 15 and later), the back gesture, the lifecycle, saving files (SAF), the bridge to Pro, the bar colors for the light or dark display
Android	Pro	app/src/io/github/hiroshitsu/qcalc/Pro.java	215	The state of QCalc Pro (owned, pending, earlier user, the test build's lock) and whether the build has billing (loads PlayBilling by its class name)
Android	Engine	app/src/io/github/hiroshitsu/qcalc/Engine.java	357	Starting, watching and restarting the engine, matching requests and replies, saving and restoring ans
Android	RatesUpdater	app/src/io/github/hiroshitsu/qcalc/RatesUpdater.java	164	Downloading, checking and replacing the exchange rates
Android	StateStore	app/src/io/github/hiroshitsu/qcalc/StateStore.java	123	Saving the history, the settings, the expression being typed and the like to files
Android (billing)	PlayBilling	app/src-billing/io/github/hiroshitsu/qcalc/PlayBilling.java	296	The connection to the Google Play Billing Library 8.3.0, the price, the purchase screen, acknowledging, asking for purchases (only in builds with billing)

Layer	Module	File	Lines	Role
Engine	Calculation engine (bridge)	bridge/qalc-engine.cc	1,647	The JSON protocol, calling libqalculate, the beginning and the end of long results, the values of graphs (2D and 3D), managing ans, the watchdog, protecting the definitions files, adding the Japanese units and trace(), closing message holds, checking the spelling of inserted units
Engine	Calculation core	libqalculate 5.13.2, GMP 6.3.0, MPFR 4.2.2, libxml2 2.15.4 (sources in third_party/)	—	The official sources, statically linked as they are
Build	Unpacking the sources	fetch-sources.sh	29	Checks the sources in third_party/ and unpacks them into src/ (downloads only what is missing)
Build	Building the engine	build-engine.sh	83	Builds the 4 libraries and the engine for arm64
Build	Common part	build-common.sh	46	The SDK location, the version, compiling the Java code (app/src only, not the billing code in app/src-billing), preparing the manifest (used by the next two)
Build	Test APK	build-apk.sh	69	Builds the APK of the test build (.debug), checks the name of the activity it starts, and signs it with the test key
Build	App Bundle	build-aab.sh	82	Builds the .aab for Google Play (without billing) and signs it with the upload key
Build	License texts	make-licenses.py	92	Makes app/assets/licenses.js (when a library version changes)
Build	Android Studio settings	settings.gradle, build.gradle, app/build.gradle, gradle.properties	138	Builds the same sources with Gradle. qcalc.billing=true adds app/src-billing and the Billing Library (build and maintenance manual 5.11)
Build	Version	version.properties	4	versionName and versionCode (shared by all builds)

Layer	Module	File	Lines	Role
Build	Icons	icon/make-icons.sh, icon/draw-icon.py	59, 226	icon/draw-icon.py draws the icon (qcalc-icon.svg in color and qcalc-icon-mono.svg in one color), and icon/make-icons.sh makes the icons of every size in app/res (WebP) from them (only when the icon changes)
Publishing	Public pages	docs/	—	The introduction page and the privacy policy on GitHub Pages (English and Japanese)
Publishing	Store listing	store/	—	The Google Play listing texts, icon, feature graphic and screenshots (English and Japanese)

2.4 Screens and transitions

The app has one main screen and, over it, 11 panels, the menu and action sheets. Everything switches inside one HTML page; the page is loaded again only when the language changes.

Screen	Opened from	Main contents
Main screen	Starting the app	The top bar (☰ and the buttons for angle, approximation and number base), the history (how to start and examples while it is empty), the input field (“Undo” right after an expression was cleared), the 8 always-shown keys, the 4 key tabs, 25 keys
Menu	☰	13 items (Examples and tips, the 6 panels below, QCalc Pro [only in builds with billing and in test builds], Settings, Language / 言語, Export history (CSV), Clear history, About this app)
Examples and tips	The menu, the button on the first screen, About this app	69 expressions in 12 groups (a tap closes the panel and puts the expression into the input field) and 12 tips

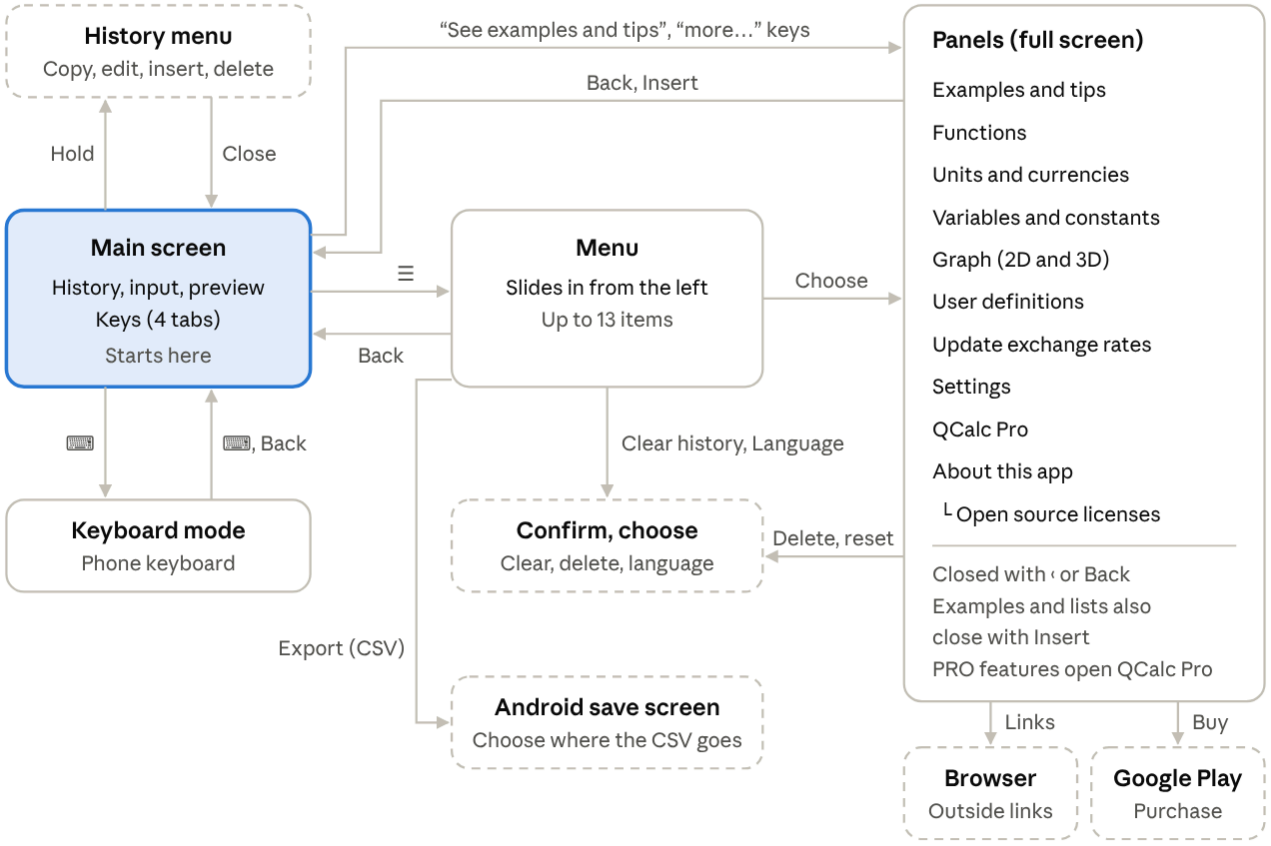
Screen	Opened from	Main contents
Functions	The menu	Category buttons, search (best match first; Japanese names too, on the Japanese screen), syntax, description, “Insert”. From “more…” on the Calculus and Matrix pages, it opens at that category
Units and currencies	The menu	Starts with the currencies. On the Japanese screen, search works with Japanese names too. The category “Japanese Units” has tsubo and 9 more. “Convert” (to that unit) and “Insert” (text that reads as that unit)
Variables and constants	The menu	Categories, search, values, “Insert”
Graph	The menu	2D or 3D. 2D: expressions (up to 6) and the x range; 3D: expressions (up to 4), the x and y ranges and the grid. Draw, zoom, reset, tap for coordinates
User-defined variables and functions	The menu	The saved definitions (insert, delete), adding variables and functions
Update exchange rates	The menu	When the rates were last updated, “Update now”
Settings	The menu	20 settings for language, calculation, display and input (including light or dark, the text size and the color theme), “Reset all settings”
QCalc Pro	The menu (builds with billing and test builds), the notice of a feature marked PRO	Whether Pro is unlocked, the 5 Pro features, “Buy QCalc Pro (price)” and “Restore a purchase”. In test builds, a switch that locks the features to try the app without Pro

Screen	Opened from	Main contents
About this app	The menu	Versions, the phone, examples, privacy, licenses, links to the source code (GitHub) and the Qalculate! website
Open source licenses	About this app	The names, versions, copyrights and full license texts (opened with a tap) of the app and the 7 bundled pieces of software (8 in all)
Action sheet	Touch and hold in the history, confirming deletions, choosing the language, the notice of a Pro feature	Choices and “Cancel”. It also opens over panels

The back gesture closes what is on top first (action sheet → menu → panel → keyboard mode). When nothing is open, the app moves to the background, and the expression being typed stays.

From the main screen and the menu to 11 panels; Back closes what is on top first

Dotted frames: sheets and screens outside the app. Examples and functions also open from the main screen



Back closes what is on top first; with nothing open, it moves the app to the background (the expression stays). Another language reloads the page in that language, on the main screen (history and expression kept).

Screen transitions · main screen, menu, panels and sheets

Of the 13 menu items, 10 open panels (full screen); “Clear history” opens a confirmation sheet, “Language / 言語” a sheet to choose the language, and “Export history (CSV)” Android's save screen. When Pro can be bought and is not owned, choosing a feature marked PRO (a precision above 1000 digits, 3D graphs, color themes, export) shows a sheet with the reason and “About QCalc Pro”. The buy button of the QCalc Pro panel opens Google Play's purchase screen. Deleting one line of the history asks for no confirmation (it can be brought back with “Undo” for 5 seconds; a cleared history, after a confirmation, for 6 seconds); deleting a user definition and resetting the settings ask first. “About this app” opens the open source licenses panel over itself, and the privacy policy, the source code and the Qalculate! website in the browser.

Screenshots of the real screens are in the user manual: 4.2 (the calculator, dark theme, landscape, the first screen, undo), 4.3 (the Functions keys, the history menu), 4.4 and 4.5 (the Calculus and Matrix keys, “more…”), 4.7 (examples), 4.8 (the menu and its panels, Japanese units, text size, About this app, 2D and 3D graphs) and 4.9 (settings).

2.5 Screen layout and keys

From the top, the main screen has four parts: the top bar, the history, the input field and the keys. Only the history changes its height.

Part	Size	Contents
Top bar	52 px high	☰ (menu), the title, 3 buttons (RAD/DEG/GRA, AUTO/EXACT/APPROX, DEC/HEX/BIN/OCT; in Japanese 自動/厳密/近似 and 10進/16進/2進/8進)
History	All the remaining height	Expressions with their results, the newest at the bottom. Results start with “=” or “≈”. Matrices with two or more rows are tables. While the history is empty: how to start, 4 examples and an “Examples and tips” button. A long result scrolls in its own box when it is taller than 70 % of the history (1.2.0). A result longer than 450,000 bytes shows its beginning, “…” and its end (1.3.0)
Input field	As much as needed	The parsed expression (1 line), the expression (up to 110 px × the text size factor, then it scrolls), the preview (up to 4.6 lines; a long result scrolls inside and can be read to the same digits as in the history), warnings and errors (up to 3.2 lines), and suggestions (one line that scrolls sideways, shown only when there are any; 1.4.0)
Keys	Always-shown keys 38 px + tabs 27 px + keys 52 px × 5 rows	34 px, 24 px and 46 px when the screen is 640 px high or less. At 360 × 740 the first screen's explanation fits in the history

In landscape with a height of 560 px or less, the history and the input field take the left half and the keys the right half (each side is half of the screen, and a long expression does not change the widths; fixed in 1.4.0). In keyboard mode (⌨) the 5 × 5 keys and the tabs are hidden and only the always-shown keys stay.

From Android 15, apps with targetSdk 35 or later are drawn behind the system bars (QCalc has targetSdk 36). So MainActivity moves the page in by the size of the status bar, the navigation bar, display cutouts and the keyboard, and puts a band of the top bar's color behind the status bar. Up to Android 14, the system keeps the page clear of the bars as before.

Always-shown keys (the top row, the same on every tab)

x	,	a=b	→	ans	⌨	◀	▶
Types x	Types “, ”	Types “ = ” (the equals sign of an equation; the big = key calculates)	Types “ to ” (conversion of units and number bases)	Types ans	Switches to the phone's keyboard	Moves the cursor left (to the start of a selection)	Moves the cursor right (to the end of a selection)

Key tabs: below them are four tabs, “123”, “Functions”, “Calculus” and “Matrix”; a tap changes the 5 × 5 keys below to that page. The chosen tab is colored. The app starts on “123” (the chosen tab is not saved).

The “123” tab

Column 1	Column 2	Column 3	Column 4	Column 5
(	)	%	⌫	AC
7	8	9	÷	x^y (^)
4	5	6	×	√ (√())
1	2	3	−	π
0	.	$\times 10^n$ (E)	+	=

The “Functions” tab

Column 1	Column 2	Column 3	Column 4	Column 5
sin	cos	tan	⌫	AC
asin	acos	atan	ln	$\log_{10}$ (log10())
x^2 (^2)	x^{-1} (^(−1))	$\sqrt[3]{}$ (cbrt())	e^x (exp())	n! (!)
|x| (abs())	e	i	°	mod
solve	∫ (integrate())	d/dx (diff())	Σ (sum())	=

The “Calculus” tab (in parentheses: the text typed; ▏ is where the cursor goes)

Column 1	Column 2	Column 3	Column 4	Column 5
d/dx (diff(▏))	d²/dx² (diff(▏, x, 2))	∂/∂y (diff(▏, y))	⌫	AC
∫ (integrate(▏))	$\int_a^b$ (integrate(▏, 0, 1))	lim (limit(▏, 0))	Σ (sum(▏, 1, 10))	Π (product(▏, 1, 10))
dsolve (dsolve(diff(y, x) = ▏))	max/min (extremum(▏))	solve (solve(▏))	∞	y
(	)	x^y (^)	√ (√(▏))	π
sin (sin(▏))	ln (ln(▏))	e	more… (opens the function list at “Calculus”)	=

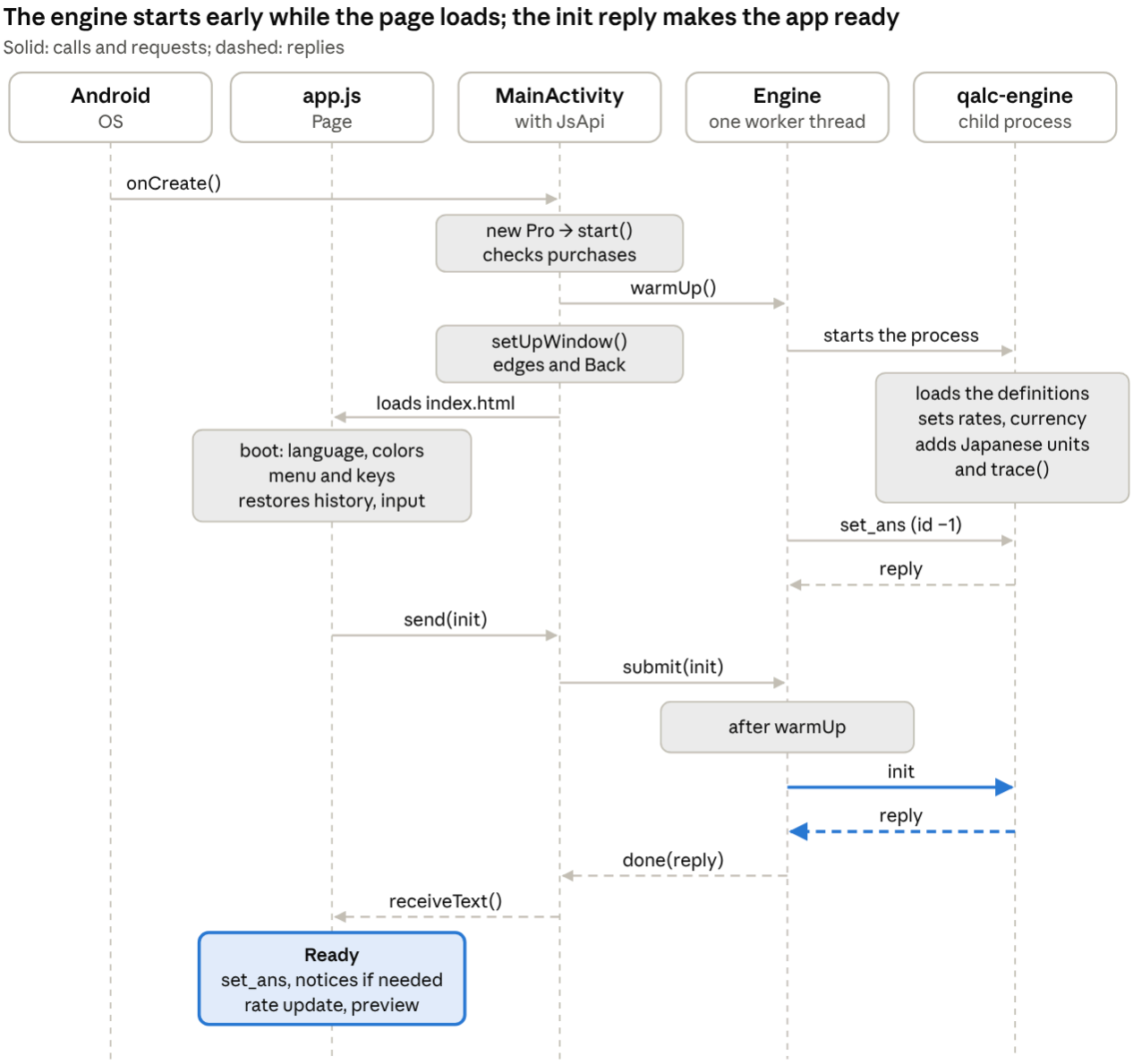
The “Matrix” tab (with digits, so that [1, 2; 3, 4] can be typed without changing the tab; the “,” between elements is in the top row)

Column 1	Column 2	Column 3	Column 4	Column 5
[	]	; (“; ” between rows)	⌫	AC
7	8	9	det (det(▏))	A^T (transpose(▏))
4	5	6	A^{-1} (^(−1))	×
1	2	3	−	+
0	.	I_n (identity(▏))	more… (opens the function list at “Matrices & Vectors”)	=

A function key types “name()” and puts the cursor inside the parentheses. A Calculus template (integrate(, 0, 1) and the like) puts the cursor at its first argument. With text selected, the key wraps the selection in the function or template. ⌫ deletes a function or template that is still empty at once. In an empty input field, ÷ × + ^ x^2 x^{-1} n! → put ans in

front (not −, so that a negative number can be typed). ⌫ and ◀▶ repeat when held: after 0.42 seconds, every 0.07 seconds. The Japanese screen shows the same keys, with 極値 in place of max/min and ほか… in place of more….

2.6 Main flows



Sequence diagram · from start to ready

At start, MainActivity first creates Pro and calls start() (a build with billing connects to Google Play and checks the purchases), then calls Engine.warmUp() before the page loads, which starts the engine process. setUpWindow() then makes the page's container and registers the handling of screen edges (insets) on Android 15 and later and the back callback (OnBackInvokedCallback) on Android 13 and later. While the engine loads its definitions and exchange rates, the page (app.js) picks its texts in the saved language (English by default), reads the state of Pro with QalcNative.proInfo, builds the colors, the

menu and the keys, and restores the history and the expression being typed. If an ans was saved, Engine restores it with set_ans (id −1) right after the engine starts. It is read with a precision raised to fit its digits, so approximate values come back with all their digits (1.2.0; before, they were rounded to about 38 digits). Engine handles requests one at a time on one worker thread, so the page's init is handled after this preparation.

The app is ready when the reply to init arrives. The page then:

- restores the last result with set_ans when the engine has no ans (ans_defined is false) and the history is not empty;
- says that the user definitions were moved aside when their file could not be read (defs_error);
- updates the exchange rates when the setting “Update exchange rates automatically (every 7 days)” is on, the rates are older than 7 days and the last attempt was at least 6 hours ago;
- calculates the preview when the input field has an expression.

If init fails, a message such as “Could not start the calculation engine” is shown; tapping it tries again. Calculating, stopping and recovering are detailed in 3.5 and 3.6, and the exchange-rate update in 3.11.

The suggestions added in 1.4.0 work inside the page only. The page loads the lists of functions, variables and units (catalog) 2.5 seconds after the app is ready, and at every input it makes the suggestions from the place of the cursor (3.24). No command was added between the app and the engine.

2.7 Data design

All saved data is in the app's internal storage (filesDir and SharedPreferences) and never leaves the phone. In the table, “state/” is filesDir/state/ and “qalculate/” is filesDir/qalculate/.

Data	Where	Format	Written
History	state/qalc.history.json (the same in the WebView's localStorage)	A JSON array of up to 300 items. The newest results keep all their digits up to 1,000,000 characters in all; older long results keep their first 2,000 characters and "…" (1.2.0). A result longer than 450,000 bytes keeps its beginning, its end (tail, 20,000 bytes) and its number of digits (1.3.0)	0.3 seconds after a calculation, and when the app goes to the background

Data	Where	Format	Written
Settings	state/qalc.settings.json (same)	A JSON object of 20 settings (text_size added in 1.1.0, the language lang and the color theme theme in 1.3.0, light or dark appearance in 1.4.0; when missing: normal, English, blue, the same as the phone. MainActivity also reads appearance at start, so that the bars have the right colors before the page loads. lang only mirrors the language in use, which the next row decides)	At every change

Data	Where	Format	Written
Language	state/qalc.lang.json (same)	A JSON string, "en" or "ja": the language the user chose with "Language / 言語". When missing: English. Kept apart from the settings so that only a language the user chose counts (test builds of 1.3.0 saved one with the settings). 1.3.0	When a language is chosen
Expression being typed	state/qalc.draft.json (same)	A JSON string	0.5 seconds after typing, after a calculation, when the app goes to the background
Graph expressions, ranges and kind	2D: state/qalc.plot.exprs.json, qalc.plot.min.json, qalc.plot.max.json. 3D: qalc.plot3d.exprs.json, qalc.plot3d.xmin.json, qalc.plot3d.xmax.json, qalc.plot3d.ymin.json, qalc.plot3d.ymax.json, qalc.plot3d.res.json (grid). The last kind opened: qalc.plot.mode.json; the drawing-finished marks: qalc.plot.ok.json, qalc.plot3d.ok.json	JSON	On "Draw" (the kind when it is switched, the marks before and after drawing)
Last automatic exchange-rate update	state/qalc.rates.try.json	A JSON number (UNIX seconds)	Before an automatic update

Data	Where	Format	Written
ans	The key “ans” of the SharedPreferences “engine”	An expression that can be read again (up to 500,000 characters, raised from 100,000 in 1.2.0. An approximate value is written as interval(lower, upper), about 400,000 characters at 200,000 digits)	At every calculation (ans event)
State of QCalc Pro	The SharedPreferences “pro”	owned (bought), price (the price to show), pre_pro, era and legacy (whether the user used QCalc before Pro), test_locked (the test build's lock). 1.3.0	At start, and whenever the purchases are checked
Exported history	Where the user chooses on Android's save screen	CSV (UTF-8 with a byte order mark). Columns: date and time, expression, result, number. 1.3.0	On “Export history (CSV)”. The app does not remember where
User-defined variables and functions	qalculate/definitions/variables.xml, functions.xml (also units.xml, datasets.xml)	libqalculate's definitions XML	At every definition and deletion
Unreadable definitions files	qalculate/broken-definitions/ (with the time after the name)	The original file	Moved aside when the engine starts
Exchange rates	qalculate/eurofxref-daily.xml, btc.json, rates.json	As the sources send them	At every update

Data	Where	Format	Written
Temporary variables (x = 5 and the like)	Only in the engine's memory	—	Not saved (gone when the engine stops)

Every file is written to a temporary file that then replaces the old one by renaming, so that a forced stop leaves either the old or the new content. The manifest has allowBackup=true, so these files are included in Android's automatic backup.

2.8 External interfaces

The app talks to the outside through the 3 exchange-rate sources and 11 Android features, and, in builds with billing, Google Play's billing. The sources and the saved file names are the ones libqalculate gives, the same as in the official version.

Exchange-rate sources

No.	Source	URL	Saved file	Contents
1	European Central Bank (ECB)	https://www.ecb.europa.eu/stats/eurofxref/eurofxref-daily.xml	eurofxref-daily.xml	The main currencies against the euro
2	Coinbase	https://api.coinbase.com/v2/prices/spot?currency=EUR	btc.json	Bitcoin
3	fawazahmed0 currency-api	https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/eur.json (if it fails, https://latest.currency-api.pages.dev/v1/currencies/eur.json)	rates.json	Many other currencies

All of this is HTTPS downloading, with the User-Agent “QCalc/1.4.0 (Android; libqalculate 5.13.2)” (the version is the app's versionName; 1.4.0-debug in the test build). Connecting stops after 15 seconds and reading after 20 seconds; a reply longer than 1,000,000 bytes is thrown away.

Android features

Feature	Use
Internet (INTERNET permission)	Downloading exchange rates

Feature	Use
Vibration (VIBRATE permission)	An 8 ms vibration when a key is pressed (can be turned off in the settings)
Clipboard	Copying results and expressions. Up to Android 12 the app shows “Copied” (from 13 the system shows it). When Android refuses a text as too long: “Too long to copy”
Browser (ACTION_VIEW)	Opens only http and https links in the browser (privacy policy, source code, the Qalculate! website, where licensed software comes from)
Region and time offset	Passes the currency of the phone's region (JPY in Japan) and the offset from UTC to the engine as environment variables when it starts
Dark theme	Reads the phone's setting and switches the colors of the page and the status bar (the page is made again when it changes). When the setting “Light or dark” is Light or Dark, it overrides the phone's setting; the page tells MainActivity with setAppearance, and the bars get the same colors (1.4.0; 3.25)
Android System WebView	Needed to show the page. When it is not available, a message asks to update or turn it on
System bars and display cutouts (Android 15 and later)	The page is drawn behind the bars, so it is moved in by the WindowInsets (bars, cutouts, keyboard). The bar colors come from views placed behind them
Back gesture	Received with OnBackInvokedCallback from Android 13 and onBackPressed up to 12; the page (qalcBack) closes what is on top. Without an answer within 0.8 seconds, the app moves to the background
Saving files (Storage Access Framework)	Exporting the history. ACTION_CREATE_DOCUMENT opens Android's save screen and the file is written where the user chose (no storage permission)

Feature	Use
Google Play billing (only builds with billing)	With the Google Play Billing Library 8.3.0: the price of the one-time product qcalc_pro, the purchase screen, acknowledging purchases and asking for them. The Billing Library adds the permission com.android.vending.BILLING

The engine is a child process of the app and gets the environment variables HOME, QALCULATE_USER_DIR (filesDir/qalculate), TZ (LOC-9:00 in Japan), LANG=C.UTF-8 and QALC_CURRENCY. The protocol is in the detailed design, 3.3.

2.9 Error handling

There are three rules: always reply, even with an error, so that the page never waits; throw a broken engine away and start a new one; never overwrite good data with a broken result or file.

Situation	Detected by	What the app does	What the user sees
A wrong expression	libqalculate's messages (error, warning)	Keeps the result with its messages in the history when there is a result; nothing when the result is empty	Red or orange text; the common ones translated on the Japanese screen
Warnings and errors no longer shown (a problem of libqalculate 5.13.2)	After a definite integral with an infinite bound, libqalculate leaves one message hold open. The bridge program looks for open holds before each command and before collecting messages	Closes all open holds. Messages found right after a calculation are returned as that calculation's messages	Warnings and errors as usual (in the official qalc, “Division by zero” and the like stop appearing after that)
Timeout (a calculation that can be interrupted)	libqalculate's calculation does not end within the time limit	Throws away the partial value and returns timeout:true	“The calculation took too long and was stopped” (“…” in the preview)

Situation	Detected by	What the app does	What the user sees
A calculation that cannot be interrupted	The watchdog in the engine (the limit + 3 seconds)	Returns a timeout and exits. The next request starts the engine again and restores ans	Same
A result that is too long	The result is longer than 450,000 bytes in UTF-8 (a matrix of three or more 200,000-digit numbers and the like; from 1.3.0 the same in the preview)	Returns the beginning, the end (tail, 20,000 bytes) and the numbers of characters and digits (truncated:true). Such a result is not inserted into expressions	The beginning, "…" and the end, with "The middle of this long result is not shown (N digits)"
Too long to copy	Android refuses the text for the clipboard (about 1 MB at a time; a complex number with 200,000-digit parts and the like)	Catches the exception; the app keeps running	"Too long to copy"
The engine does not answer	The Java side's waiting limit (the limit + 10 seconds)	Kills the engine. The request is not sent again	Same as a timeout
The engine crashes	Its output closes while it handles a request	The request is not sent again (the same expression would crash it again). The next request starts the engine	"The calculation engine stopped unexpectedly. Try a simpler expression."
Writing fails	The engine had stopped before the request was sent	Starts the engine and sends the request once more	Nothing
The engine cannot start	Starting or writing fails twice	Returns an error	"Could not start the calculation engine"; tap to try again

Situation	Detected by	What the app does	What the user sees
Stopped by the user	A tap on “Calculating…”	Stops the engine	“The calculation was stopped”
Out of memory	An exception that cannot be caught (std::terminate)	Returns an error and exits	“Not enough memory for the calculation”
An expression nested too deeply	Parentheses deeper than 300 or more than 1,000 ^	Does not calculate (to avoid a stack overflow)	“The expression is nested too deeply”
A reply that cannot be read or does not arrive	A JSON parse error, the page′s final timer (at least 120 seconds)	Ends the waiting request as failed	“Could not read the reply of the calculation engine” and the like
The page′s renderer stops	onRenderProcessGone	Makes the page again; not a second time within 15 seconds	The history and the expression stay. The second time: “The screen stopped again and again. Close the app and open it again.”
Exchange rates cannot be downloaded	A network error or a failed check, per source	Replaces only what was downloaded. A broken download never overwrites	The names of the sources that failed, or “Could not download the exchange rates. Check the network connection.”
A broken definitions file	It cannot be read as XML at start	Moves it to broken-definitions/, then loads	“The saved user definitions could not be read, so their file was moved aside”

Situation	Detected by	What the app does	What the user sees
Saving a definition the engine cannot read	error or warning from libqalculate's parsing	Does not save it	“Check the value (…)”
A wrong graph range	The range cannot be calculated as a number within 2 seconds, or minimum ≥ maximum (invalid range, invalid x range, invalid y range)	Does not calculate the graph	“Check the range of x”, “Check the range of y”
A graph takes too long	12 seconds for 2D, 15 for 3D (the engine's watchdog 3 seconds later)	3D returns the rows it finished and leaves the rest empty (complete:false). No values when the watchdog answered	“The calculation took too long: only part of the graph was calculated” or “The calculation took too long and was stopped”
The page stopped while drawing a graph	The drawing-finished mark is set to false before drawing and true after; it is still false at the next opening	Opening the panel does not draw by itself (so that an expression that crashes the page is not repeated)	“Press ‘Draw’ to draw the graph”
A Pro feature without Pro	Pro can be bought (billing is play) but is not owned, and a feature marked PRO was chosen	Precision becomes 1000, 3D stays 2D, the color theme stays blue, nothing is exported; a sheet shows the reason and “About QCalc Pro”	From the sheet to the QCalc Pro panel

Situation	Detected by	What the app does	What the user sees
Google Play cannot be used	Billing replies such as BILLING_UNAVAILABLE, SERVICE_UNAVAILABLE and NETWORK_ERROR	The state of Pro stays as it was (the saved owned is used); asked again at the next start or return	“Google Play is not available on this device”, “Could not connect to Google Play. Try again later.”
Pro cannot be bought	ITEM_UNAVAILABLE and other reply codes (not the user's cancel)	Pro stays locked	“QCalc Pro cannot be bought right now”, “The purchase could not be completed (code)”
A pending payment	The purchase state is PENDING	Unlocks Pro when the payment is done (PURCHASED) and acknowledges it	“The payment is pending. QCalc Pro will be unlocked when it is complete.”
No purchase found	“Restore a purchase” finds no purchase in Google Play	Changes nothing	“No purchase of QCalc Pro was found”
A file cannot be saved	An exception while writing, or the app was made again while the save screen was open and the content was lost	Tells the page with a saved event (ok:false); nothing is shown when the user cancelled	“Could not save the file”

2.10 Security and privacy

The app loads no outside pages and sends no calculations. What must be protected is the data on the phone and the upload key.

What	Measures
The page (WebView)	Shows only the app's own pages under file:///android_asset/. File access and content access are off. Other links open in the browser, http and https only
Text on the page	Engine results and user input are set with textContent, never read as HTML
JavaScript bridge	Only the bundled page can call it. Requests are checked as JSON, and those with line breaks are refused. Saved names are limited to lowercase letters, digits and ".-", *up to 64 characters. An exported file must have a name of letters, digits and ".-"* up to 100 characters, the type text/csv and at most 16 × 1024 × 1024 characters, and the user chooses where it goes on Android's save screen
The engine	Built with --disable-insecure, so command(), which would run outside commands from an expression, is disabled. It runs with the app's rights and reads and writes only filesDir/qalculate/
Network	Only HTTPS downloads from the 3 exchange-rate sources and, in builds with billing, Google Play (Billing Library). No analytics, no ads. The sources receive an ordinary HTTPS request (IP address, User-Agent and the like) and nothing about the user
Data on the phone	All in the app's own storage. With allowBackup=true it is included in Android's backup (Google account)
Clipboard	Copied results can be read by other apps (as Android works)
Permissions	Only INTERNET and VIBRATE (builds with billing also get com.android.vending.BILLING from the Billing Library)

What	Measures
Signing key (upload key)	Google keeps the app signing key that signs the Google Play build (Play App Signing). The developer keeps the upload key signing/qcalc-upload.jks and its password; neither is published or put into the source package or the repository (excluded by .gitignore). If it is lost or leaked, a new upload key is requested in the Play Console. The test key is used only for the test build (.debug)
Privacy policy	https://hiroshitsu.github.io/qcalc/privacy.html (English and Japanese), linked from “About this app” and the Google Play listing. Data safety is declared as “no data collected, no data shared”
Licenses	The app is GNU GPL v2 or later (with a permission to link the Google Play Billing Library); the engine as a whole is GPL v3 or later (MPFR is LGPL v3 or later). The full license texts are shown in the app, and the sources of the libraries in the engine are in third_party/ in the repository
Checking purchases (QCalc Pro)	Google Play keeps the record of purchases; the app asks at every start and whenever it comes back to the front. A paid purchase (PURCHASED) unlocks Pro and is acknowledged if it was not yet (retried at the next check if that fails). A refunded purchase disappears at the next check. The owned flag saved on the phone is overwritten by each check. There is no server-side verification